

Requirement Management Field Manual

Memorize the whole deck the easy way: one story, the exact exam points, and a memory hook on every topic.

Module **MMC-01a** Lecturer **Prof. Dr. M. Górka** Source **2.pptx · 151 slides** THD · NuW · AI for Smart Sensors



Meet Pip the parrot — your memory anchor

A kid named Mira walks into a robot shop and says “I want a robot pet.” Behind the counter sits Pip, a parrot whose one job is to repeat the wish back perfectly — because if Pip mishears a single word, the workshop in the back builds the wrong robot.

Every slide answers one question: “How does Pip turn a fuzzy human wish into exact, written, testable orders — and keep them straight for the product's whole life?” Keep Mira (customer) → Pip (requirement) → workshop (supplier) in your head and the chapter tells itself.

★ **MASTER KEY** → **basket** → **scale** → **index cards** → **ladder** → **checklist** → **catch changes** = **Collect** · **Assess** · **Develop** · **Tier-down** · **Verify** · **Changes**

00 · THE SHAPE OF THE WHOLE CHAPTER

See the skeleton before the bones

Requirement Management is step 2 of System Engineering:

1 Basics → **2 REQUIREMENTS ★** → 3 Risk → 4 Change → 5 Ideation → (6 Design) → (7 Test) → (8 Project Setup)

The chapter solves requirements in a fixed order — the **6-phase process**. Learn this one picture-story and you already own the structure of the exam; everything else just bolts onto one of these six scenes:



basket → **scale** → **index cards** → **ladder** → **checklist** → **catch the changes**.

Collect → **Assess** → **Develop** → **Tier-down** → **Verify** → **Changes**.



The Memory Movie

Don't memorize sentences — your brain hates random letters. Instead **close your eyes and play this 6-scene movie** of Pip building Mira's robot pet. Each picture is a phase of the process, in order, so when you replay the movie the exam answer just appears. Read it 3 times, then try to replay it with your eyes shut.



SCENE 1

Pip walks around with a **basket**, scooping up every wish, law and old note. → (1) **COLLECT requests**.



SCENE 2

He tips the basket onto a **judge's scale**: relevant? important? what kind? → (2) **ASSESS requests (the Kano judge lives here)**.



SCENE 3

He writes each survivor as a crisp **index card** — short, exact, testable. → (3) **DEVELOP requirements (SMART · testable · linked)**.



SCENE 4

He carries the cards **down a ladder**, handing smaller orders to each workshop floor. → (4) **TIER DOWN to component level**.



SCENE 5

He ticks a **checklist** as the workshop proves each card was built right. → (5) **track VALIDATION / VERIFICATION**.



SCENE 6

Someone changes their mind — Pip catches it and **updates every card**. → (6) **follow up of CHANGES**.



Replay in one breath: **basket → scale → index cards → down the ladder → tick the checklist → catch the changes**. That's the whole chapter's skeleton — no letters needed.



The two binders Pip carries (the Core Documents)

Picture two ring-binders on the counter — this is the most common exam pair:



Red WISH binder = PRD

German *Lastenheft*. The **customer's** “**what**” — the sum of all customer requirements; used to request offers.



Green PROMISE binder = Impl. Spec

German *Pflichtenheft*. The **supplier's** “**how & with what**.” Sign it → development starts.

Picture-hooks for the tricky lists

Same trick for every list that wouldn't stick — turn it into one strong, weird image:



Why RM? (tree-swing)

Everyone draws a different swing from the same words. That chaos is **why Pip exists**: one common understanding, no field disasters.



Kano's 5 shelves

 bread (basic/must — noticed only when gone) ·  dial (performance/should — more = happier) ·  gift (delighter/could — wow!) ·  grey box (indifferent) ·  stink bomb (reverse/don't — worse).



SMART

A brain in a smart-watch: **Specific · Measurable · Achievable · Reasonable · Time-bound**.



Why specific

"The man on the mountain with the telescope" → 5 different drawings. Language is ambiguous → **fix with IF-THEN code**.



Not TOO specific

A bicycle hidden in a TV box — they fixed the **problem** (shipping damage), not the package → a clever solution emerged.



"Linked" disaster

A broken bridge with a 28 cm step (North-Sea zero vs Mediterranean zero) → **unlinked requirements smash together**.



System Wheel

A wagon wheel: hub = mechanics/E-E/software · inner ring = 7 "DNA" datasheet props · outer ring = 10 extended properties.



Tier-down bouncer

A bouncer at each door: **acceptable? → in. Not? → can I tweak it? yes: modify / no: toss**.



Analysis paralysis

A man frozen forever at the planning desk, never starting the build. **Don't be him**.



ReqIF

A universal plug (XML) letting two requirement tools — and customer↔supplier — **exchange requirements**.



How to actually lock it in: (1) read a scene, (2) **shut your eyes and re-see the picture**, (3) say the real meaning out loud. Do the 6-scene movie + 10 picture-hooks twice today and once tomorrow — then test yourself with the **flashcards**. Seeing beats reading; testing beats re-reading.

Taught like a story — with the points you must write

01 What *is* a requirement?

STORY

Mira says “robot pet.” That's a wish, not a buildable order. Pip's job is to turn it into a **requirement**: an exact written target for one property that already accounts for all conditions (outdoors? quiet at night?). One robot needs **hundreds to tens of thousands** of these cards.

WRITE IN EXAM — DEFINITION

- **Requirement** = an accurately phrased development target for a property of the system/component, **considering all relevant conditions**.
- **Amount**: several hundred to **(ten-)thousands** (grows over time, e.g. autonomous cars).
- Defined in standards: **ISO/IEC/IEEE 29148** (software req. engineering), **ISO/IEC 25000 (SQuaRE)**; part of maturity models **CMMI, SPICE (ISO/IEC 15504), ISO/IEC 12207**.
- Lives inside **ItO – Idea to Offer**: customer requirement → ItO → offer → product (purchasing procures in parallel; after test/validation & industrialization → release).



A requirement is **one exact card for one property** — not a vague wish. Software rulebook number to anchor: **29148**.

02 Why do we need RM? (the tree-swing)

STORY

The famous **tree-swing cartoon**: the customer explains a swing, sales describes another, the designer draws a third, the workshop builds a fourth — and what the customer *really* needed was something else entirely. That mess is exactly what Pip prevents.

WRITE IN EXAM — WHY RM

- RM identifies the necessary requirements **from (super)system level down to component level** and **maintains them over the whole lifecycle**.
- RM is the **enabler** to: increase **customer satisfaction** · get a **common understanding** · avoid **safety/field (quality) issues** · avoid **late changes & cost impacts** · feed **LeLe**.
- → RM is the **essential discipline in systems design** — **if the targets are set wrong, the whole product is wrong!**
- **Challenges (3)**: clear & distinct definition · completeness (missing reqs found too late) · inaccurate interpretation. → **LeLe is important!**

🗨️ Same words, five different swings. RM = one common picture.

03 The two binders (Core Documents)

🗨️ STORY

Pip carries two binders. Mira hands over the ■ **red WISH binder (PRD)** = everything *she* wants. The workshop reads it and writes back the ■ **green PROMISE binder (Implementation Spec)** = exactly *how* and *with what*. When Mira signs the green binder, building begins.

📖 WRITE IN EXAM – PRD (LASTENHEFT)

- **Sum of all requirements of a commercial customer** (not necessarily the end user); standards **DIN 69901-5 / IEEE SRS 29148**.
- Used in the **nomination phase** to request technical & commercial **offers** from suppliers.
- **Content:** intro · current state · target (main reqs) · interfaces · **functional** · **NFR** · system architecture · expected scope of delivery · acceptance criteria.
- **Internal** dev → detailed & tiered down. **External** dev → generic (property level) to **avoid specifying the solution**.

📖 WRITE IN EXAM – IMPLEMENTATION SPEC (PFLICHTENHEFT)

- The **supplier's answer to the customer's PRD**: the **“How”** (technical concept) and the **“With what”** (resources, project plan).
- **Every PRD requirement** must be covered by ≥ 1 deliverable. **Customer acceptance** → **development starts**.

🗨️ **Red = Request** (customer, “what”) · **Green = Guarantee** (supplier, “how/with what”). *Lastenheft* = load you carry → *Pflichtenheft* = duty you accept.

04 The 6-phase process (the spine)

🗨️ STORY

This is the **movie** again — Pip's six scenes. Different rulebooks describe it (IEEE SWEBOK, CMMI, IIBA/BABOK, Volere, IREB), but the lecture uses one variant with two freeze points: **PRD freeze** and **Implementation-spec freeze**. In big projects a **requirement manager** steers it.

📖 WRITE IN EXAM

- **6 phases:** (1) Collect → (2) Assess → (3) Develop → (4) Tier down → (5) Validation/Verification → (6) Changes.
- **Cascade of who:** Customer (request) → **Property Engineer** (1→x system properties) → **Developer** (x→y component requirements).

🗨️ **basket** → **scale** → **index cards** → **ladder** → **checklist** → **catch the changes**.

05 (1) COLLECT — fill the basket

STORY

Pip scoops up wishes from *outside* (Mira first!, plus law and rules) and from *inside* the shop (old notebooks, house rules). But careful — interviewing **more and more people** gives **less and less new info** (diminishing returns).

WRITE IN EXAM

- **External requests: !!!Customer!!! (the prior source!)** · Law · Standards & guidelines · Cultural · Political · Social expectations.
- **Internal requests: LeLe** · company **standards** · existing **tool kits** · **company strategy** · **CI/design** · business-dept restrictions.
- **Use cases** capture demand: “As [role] / I would like to [function] / To [reason]”; named after the goal.



Customer first, then the world outside, then your own shop. And **fewer secrets per extra interviewee**.

05a LeLe — Lessons Learned

STORY

At the **kick-off**, the team reads the old notebook so they don't repeat last project's mistakes. They keep updating it, and at the **end** hold a workshop to write the new wisdom into the standard.

WRITE IN EXAM

- Use prior-project knowledge at the **start** (kick-off) → **constantly update** during → **end workshop** transfers “what went well / what to improve” into the **new standard**. Covers **technical, PM, commercial** knowledge; document **in software**.
- **Golden rule:** the solution is **NOT the negation of the problem**. “We started the test too late” → **✗** “don't start so late” → **✓** “**start the test ≥2 weeks before release and anchor it in the project plan.**” (Same for positive feedback.)

05b Standards & “who is the customer?”

WRITE IN EXAM

- **Regional:** TGL (DDR) · DIN (German) · GB/GB-T (China). **International:** EN (European) · ISO (mechanics) · IEC (electrical/electronics). **Industry:** IATF (automotive) · VDE (electrotechnics).
- “**Who is the customer?**” **pitfall:** wrong requirements come from misunderstanding who the real customer/user is (parents choosing for the child).

06 (2) ASSESS — the judge's scale & Kano

🗣️ STORY

Pip can't build everything, so he **judges** each wish three ways: is it **relevant** here? how **important** is it? what **kind** is it? The most famous judge on the scale is **Mr. Kano**, who sorts every feature onto one of 5 shelves by how it moves Mira's happiness.

📖 WRITE IN EXAM — ASSESS BY

- **Relevance** (relevant for *this* product? e.g. not sold in Japan → ignore Japanese law) · **Priority** (which wins if not all fit, e.g. airbag > turn signal; depends on company strategy) · **Classification**.
- **Design strategies** driving priority: Design for **Customer** (Design Thinking) · **Costs** · **Quality/Reliability** · **Production/Manufacturing (DFM)** · **Service**.

📖 WRITE IN EXAM — KANO MODEL (PROF. NORIAKI KANO)

- **Basic (must)** — bread: invisible if present, furious if missing; no competitor differentiation. (*call-centre greeting*)
- **Performance (should)** — dial: **more = happier**. (*hotline waiting time*)
- **Delighter (could)** — surprise gift: unexpected → **wow**; a little gives a lot. (*live ticket-status tracking*)
- **Indifferent** — grey box: present or not, nobody cares. (*hold-music song*)
- **Reverse (don't)** — stink bomb: its presence **lowers** satisfaction. (*jargon in a hotline call*)
- Classification **shifts** by **customer**, by **price/business model**, and over **time** (delighters become basics).
- Survey: **standardised** (remove interviewer impact) + **demographics**; ask a **functional + dysfunctional** pair.

🗨️ Order of happiness: **Bread** → **Dial** → **Gift** → **Grey box** → **Stink bomb**. (*Car: must=it drives · should=horsepower · could=launch control · indifferent=underfloor-screw colour · reverse=new rust after a wash.*)

06a Classification of requests

📖 WRITE IN EXAM

- **By belonging: Product (What)** [SE focus] · **Process (How)** · **External (Rest)**.
- **By function: Functional** (prints my file) vs **NFR** (24/7, 99.9% uptime — not a feature but still expected).
- **Function attributes**: inputs/outputs · value range/dimensions · tolerance · how often executed · how fast (throughput time).

07 (3) DEVELOP — write the perfect index card

🗣️ STORY

Now Pip writes each requirement as a tiny, perfect card. Rule of thumb: **one idea per card**, written like a robot could obey it.

WRITE IN EXAM — A GOOD REQUIREMENT IS...

- **Complete** (no supplier assumptions) · **correct** · **SMART** · **testable** · **linked** · **necessary** (no “nice-to-haves”).
- **Short** · **atomic** (one req/sentence) · **active voice** (“The motor rotates the shaft” , not “is driven by” ) · **no weak words** (“really/very/roughly” — tools **ReQualize**, **DESIRE**) · **avoid “should/must”**.
- **Understandable** · **traceable** · **unambiguous & consistent** · **feasible** · **equipped with properties**.

PROPERTIES OF A REQUIREMENT

- Unique **ID** · description · **rationale/target** · **originator** · **creator** · **user** · **fit criterion** (testable) · **priority** · dependencies/interfaces · conflicts · release/maturity · linked documents · comments · **version** · **change history** · **status**.

 One card, one idea, active voice, no fuzzy words — and give each card a **passport** (ID, who, why, test, version, status).

07a SMART

STORY

Specific: “I saw the man on the mountain with the telescope” — five people draw five different pictures → fix with **IF-THEN-ELSE** code. **Not too specific:** a company asked to cut a bicycle's **shipping damage** (not “improve the box”) → result: a box that **looks like a TV**, far fewer damages. Specify the *problem*, not the *solution*.

WRITE IN EXAM

- **SMART** = **Specific** · **Measurable** · **Achievable** · **Reasonable** · **Time-Bound**.
- Natural language is **ambiguous** → use mathematical/IF-THEN descriptions and **exclude variants**.

 Brain in a **smart-watch**. + **telescope** = **be specific** / **bicycle-in-a-TV-box** = **don't over-specify**.

07b Testable & writing guideline

WRITE IN EXAM

- To be **testable** a requirement must be **specific, detailed, affordable to test, linked to a maturity level**. (“The charger needs to be efficient” = untestable — 75%? 80%?)
- Verification must be **affordable** (don't crash 10 real cars to check “mute button accessible after a crash”).

07c Linked + SCRUM

STORY

Every card holds hands with its neighbours. The  **bridge built from two countries** (North-Sea zero vs Mediterranean zero) ended with a **28 cm step** — and the fix went the wrong way, doubling it. That's what unlinked requirements do.

WRITE IN EXAM

- **Link each requirement to:** its **request** · **child & mother** reqs (hierarchy) · **neighbour** reqs (mutual impact) · **functions & NFR** · **test items** · **interfaces**.
- **SCRUM** (agile, software): **Epic** → **feature request** → **product backlog** — same idea, highly simplified; backlog rows use the role/function/reason format.

 The **28 cm bridge step** = always link your requirements.

08 The System Wheel

STORY

To not forget any property of Mira's robot, Pip spins a **wagon wheel** of properties (recap from chapter 1, used here to develop requirements as a “datasheet”).

WRITE IN EXAM — 3 LAYERS

- **Hub = realization domain:** mechanics · E/E · software.
- **Inner ring = core “DNA” (datasheet):** performance · consumption · acoustic · costs · reliability · weight · size.
- **Outer ring = 10 extended properties:** acoustical behaviour · performance & efficiency · data acquisition/access & security · design for production/service/upgrade · **safety** (ISO 26262, FuSiKo/HARA) · **EMC** (emission, immunity, ESD) · sustainability · reliability · thermal behaviour · volumetric behaviour & design (incl. **FMK** tolerance concept, crash behaviour).
- **Property engineer** = the **voice of the customer for one property**; optimises **across the system** (not one component); thinks in **system functions**; the **system engineer** decides conflicts.

 Wheel = **hub (3 domains)** + **inner ring (7 DNA)** + **outer ring (10 extended)**. Property engineer = one property's customer-voice.

09 (4) TIER DOWN — carry the cards down the ladder

STORY

Pip walks the cards **down** from supersystem → system → sub-system → component, handing each floor an order it can actually build and **own**. A loudspeaker maker **won't promise the sound inside the whole car** — he didn't build the room! Each receiver must **accept, reject, or ask to change** the card.

WRITE IN EXAM

- Tier down system → component level; the receiver (e.g. component team) must **accept / reject / feedback for change** and be able to **take responsibility**.
- **New-requirement flow:** new req → **inbox/pool** → **acceptable?** yes → add to component reqs → END. No → **modification possible?** yes → modify (or decline **with reason**) · no → **discard**.
- **Hierarchy:** Supersystem (4) → System (3) → Sub-system (2) → Component (1).



A **bouncer at each door**: fits? in. Doesn't? can I tweak it? yes→modify / no→toss. Only promise what you can build.

10 (5) Verify & (6) Changes + Analysis Paralysis

STORY

Pip doesn't run the tests himself — he just **ticks them off** as the workshop proves each card. When anyone changes their mind, he **chases the change** through every card. And he must never freeze polishing cards forever instead of letting the build start.

WRITE IN EXAM

- **(5) Validation/Verification**: all reqs must be **testable**; RM does **not verify** but **tracks** that each was **confirmed by testing** (→ Test & Verification chapter).
- **(6) Changes**: track & follow up **changes of requests/requirements** (→ Change Management chapter).
- **Analysis paralysis**: RM is vital, but don't analyse forever to be 100% perfect **instead of starting realization**.



RM **tracks the ticks**, it **doesn't do the tests**. Changes → Change Management. And **don't freeze at the desk** — start building.

11 Software, ReqIF & Summary

STORY

Thousands of cards need real tools (not Word/Excel). And two different tools — or customer & supplier — talk through one **adapter language**: ReqIF.

WRITE IN EXAM

- **Software must**: administrate · structure/sort/filter · tailor to owner · manage workflow/status · **link** · version & change-history · **branch** for products; **traceability & transparency** matter (no standard office tool!).
- **ReqIF = Requirement Interchange Format** — an **XML** format, managed by **ProSTEP iViP e.V.**, to **exchange requirements between tools and between customer & supplier**.
- **Tools**: Codebeamer (PTC) · DOORS (IBM) · JAMA. **Agile**: ClickUp · cplace · Jira (Epic → product backlog → sprint backlog).
- **Summary**: RM is the **most important step in system design** — right product, right quality, highest **customer satisfaction**; runs in **6 phases** and focuses on **properties, not technical solutions**.



ReqIF = the **universal plug (XML)** between tools and partners. *If the targets are set wrong, the whole product is wrong* — but don't drown in analysis paralysis.

YOUR PROGRESS

0 / 11 topics learned

Flashcards

Read the question, say the answer *out loud*, then flip. Cover the deck and run it twice — that's how it sticks.

All flashcards (Q/A)

1. What is a "requirement"?

An accurately phrased development target for a property of the system/component, considering all relevant conditions.

2. How many requirements does a product have?

Several hundred to (ten-)thousands, depending on complexity (and growing over time).

3. Which standards define requirements engineering?

ISO/IEC/IEEE 29148 (software) & ISO/IEC 25000 (SQuaRE); in maturity models CMMI, SPICE (ISO/IEC 15504), ISO/IEC 12207.

4. What does ItO (Idea to Offer) describe?

The core development process: customer requirement → ItO → offer → product (purchasing in parallel; then test/validation & industrialization → release).

5. What does the tree-swing cartoon show?

Everyone understands the same words differently — so RM is needed to create one common understanding.

6. What is RM the enabler of?

Customer satisfaction, common understanding, avoiding safety/field issues, fewer late changes & costs, and LeLe.

7. Why is RM essential in systems design?

If the targets (requirements) are set wrong, the whole product is wrong.

8. Three challenges for RM?

Clear definition, completeness (missing reqs found late), and inaccurate interpretation.

9. The two core documents in RM?

Product Requirements Document (PRD/Lastenheft) and Implementation Specification (Pflichtenheft).

10. What is the PRD (Lastenheft)?

The sum of all customer requirements; used in the nomination phase to request offers. DIN 69901-5 / IEEE SRS 29148.

11. What does a PRD contain?

Intro, current state, target/main reqs, interfaces, functional reqs, NFR, system architecture, scope of delivery, acceptance criteria.

12. PRD detail: external vs internal dev?

Internal: detailed & tiered down. External: generic property level to avoid specifying the solution.

13. What is the Implementation Spec (Pflichtenheft)?

The supplier's answer to the PRD: the 'How' (technical concept) and 'With what' (resources, project plan).

14. When does development start?

When the customer accepts the Implementation Specification (every PRD requirement covered by a deliverable).

15. Which bodies describe the RM process?

IEEE (SWEBOK), CMMI, IIBA (BABOK), Volere, IREB.

16. Two freeze milestones in the RM process?

PRD freeze and Implementation-specification freeze.

17. The 6 phases of the RM process?

Collect requests, Assess requests, Develop requirements, Tier down, Validation/Verification, Follow up of changes.

18. How do requests cascade by role?

Customer (request) → Property Engineer (system properties) → Developer (component requirements).

19. Prior source of external requests?

The customer (must be the prior source). Also law, standards, cultural/political/social expectations.

20. What happens with more interview partners?

Gained knowledge decreases — diminishing returns.

21. Use-case format for demand?

'As [role] / I would like to [function] / To [reason]' — named after the goal.

22. Typical internal requests?

LeLe, company standards, existing tool kits, company strategy, CI/design, business-dept restrictions.

23. What is LeLe (Lessons Learned)?

Knowledge management: use prior-project knowledge at kick-off, update throughout, transfer findings to the new standard at the end.

24. Golden rule of LeLe?

The solution is NOT the negation of the problem — write a concrete, anchored action (e.g. 'start the test ≥ 2 weeks before release, in the plan').

25. Examples of regional/international/industry standards?

Regional: TGL, DIN, GB. International: EN, ISO (mechanics), IEC (electrical). Industry: IATF (automotive), VDE.

26. Three ways to assess requests?

By Relevance, by Priority, and by Classification.

27. What does assessing by Relevance mean?

Is the request relevant for this product? (e.g. not sold in Japan → ignore Japanese law).

28. What does Priority depend on?

Which request wins if not all fit (e.g. safety) and the company strategy; requirements often impact each other negatively.

29. Name design strategies affecting priority.

Design for Customer (Design Thinking), Costs, Quality/Reliability, Production/Manufacturing (DFM), Service.

30. What is the Kano model & who made it?

A method to prioritise requests by linking features to satisfaction; by Prof. Dr. Noriaki Kano (from quality management).

31. The 5 Kano levels?

Basic (must), Performance (should), Delighter (could), Indifferent, Reverse (don't).

32. Kano: basic needs (must)?

Self-evident; only noticed when missing; absence dissatisfies, presence adds no satisfaction (e.g. call-centre greeting).

33. Kano: performance needs (should)?

Satisfaction scales with the degree of fulfilment (e.g. hotline waiting time).

34. Kano: delighter (could)?

Unexpected feature creating enthusiasm; a small increase gives disproportional benefit (e.g. live ticket tracking).

35. Kano: indifferent vs reverse?

Indifferent = little impact (hold-music song). Reverse (don't) = its presence lowers satisfaction (jargon in a call).

36. How can a Kano class change?

By customer, by price/business model, and over time (delighters become basic needs).

37. Kano survey question pair?

A functional ('how if it had MORE...?') and dysfunctional ('...if LESS/none...?') question; class read from the combination.

38. Kano: 'expect it' + 'dislike it' =?

Basic need (must).

39. Kano: 'like it' + 'neutral' =?

Delighter (could).

40. Classify requirements by 'Belonging'?

Product (What) [SE focus], Process (How), External (Rest).

41. Functional requirement vs NFR?

Functional = a dedicated feature (prints my file). NFR = an expectation, not a feature (24/7, 99.9% uptime).

42. Attributes of a function?

Inputs/outputs, value range/dimensions, tolerance, how often executed, how fast (throughput time).

43. Qualities a good requirement must have?

Complete, correct, SMART, testable, linked, necessary, short, atomic, active voice, no weak words, traceable, unambiguous, feasible.

44. Why atomic & active voice?

Atomic = one requirement per sentence. Active ('The motor rotates the shaft') beats passive ('is driven by').

45. Weak words and tools to find them?

Vague words like 'really, very, roughly'; found with ReQualize or DESIRE.

46. Key properties of a requirement?

Unique ID, description, rationale/target, originator, creator, user, fit criterion, priority, dependencies, version, change history, status.

47. What does SMART stand for?

Specific, Measurable, Achievable, Reasonable, Time-Bound.

48. Why does 'specific' matter?

Natural language is ambiguous ('the man on the mountain with the telescope'); fix with IF-THEN descriptions and exclude variants.

49. Lesson of the bicycle 'TV box' example?

Don't be too specific — specify the problem (reduce shipping damage), not the solution; a clever solution (TV-shaped box) emerges.

50. What makes a requirement testable?

It must be specific, detailed, affordable to test, and linked to a maturity level.

51. Why must verification be affordable?

Avoid impractical tests, e.g. crashing many real cars to verify 'mute button accessible after a crash'.

52. General requirement-writing guideline?

Positive not negative; avoid 'could/should'; be specific; say WHAT not HOW.

53. What must a requirement be linked to?

Its request, mother/child reqs, neighbour reqs, functions & NFR, test items, and interfaces.

54. Lesson of the bridge example?

A bridge built with North-Sea vs Mediterranean zero had a 28 cm step; the fix went the wrong way — unlinked requirements cause costly errors.

55. How does SCRUM handle requirements?

Simplified agile: Epic → feature request → product backlog → sprint backlog; rows use role/function/reason.

56. What is the System Wheel?

An overview of mechatronic-system properties: hub (mechanics/E-E/software), core 'DNA' datasheet properties, and 10 extended properties.

57. Core 'DNA' properties of the System Wheel?

Performance, consumption, acoustic, costs, reliability, weight, size.

58. Role of the property engineer?

The voice of the customer for one property; optimises across the system, thinking in system functions, and breaks function down to component reqs.

59. What is 'tier down'?

Breaking system requirements to lower (component) levels so the component team can develop and own them.

60. What must the receiver of a tiered-down req do?

Accept, reject, or feedback a request for change — and only own what it can build.

61. New-requirement decision flow?

Inbox/pool → acceptable? yes: add to component reqs. no: modification possible? yes: modify/decline with reason; no: discard.

62. System hierarchy for tier-down?

Supersystem (4) → System (3) → Sub-system (2) → Component (1).

63. RM's role in validation/verification (5)?

RM does not verify but tracks whether each testable requirement was confirmed by testing.

64. What happens in phase 6 (changes)?

Changes of requests & requirements are tracked and followed up (Change Management chapter).

65. What is analysis paralysis?

Analysing forever to be 100% perfect instead of starting realization — RM must avoid it.

66. Functions professional RM software must provide?

Administrative, structure/sort/filter, tailor to owner, workflow/status, link, change history & versioning, branching — with traceability.

67. What is ReqIF?

Requirement Interchange Format — an XML format (ProSTEP iViP e.V.) to exchange requirements between tools and between customer & supplier.

68. Example RM tools?

Codebeamer (PTC), DOORS (IBM), JAMA. Agile: ClickUp, cplace, Jira.

69. Core summary of RM?

The most important step in system design; ensures the right product with the right quality and highest satisfaction; 6 phases; focus on properties, not solutions.

3 Practice Questions + model answers

Try to answer on paper before clicking **Reveal**. The model answers show the exact points an examiner rewards.

Q1 Describe the requirement-management process. Name its phases, and explain the two core documents (PRD and Implementation Specification) and who is responsible for them.

TYPE: STRUCTURED / THEORY · ~10 MARKS

REVEAL MODEL ANSWER

✓ MODEL ANSWER

RM follows a fixed process: (1) **Collect requests** → (2) **Assess requests** → (3) **Develop requirements** → (4) **Tier down** → (5) **Validation/Verification** → (6) **Follow up of changes**, with a **PRD freeze** and an **Implementation-spec freeze**; in large projects a **requirement manager** steers it. RM exists to build the right product (the tree-swing problem) and follows the rule: *if the targets are set wrong, the whole product is wrong*.

- **PRD (Product Requirements Document / Lastenheft)**: the customer's sum of all requirements (DIN 69901-5 / IEEE SRS); used in the nomination phase to request offers. Content: current state, target, interfaces, functional + non-functional reqs, architecture, scope of delivery, acceptance criteria.
- **Implementation Specification (Pflichtenheft)**: the supplier's answer — the “How” (technical concept) and “With what” (resources/project plan); every PRD requirement must be covered. On customer acceptance, development starts.

Responsibility cascades: **Customer** (request) → **Property Engineer** (system properties) → **Developer** (component requirements). RM focuses on **properties, not technical solutions**.

Q2

Explain the Kano model. Name its 5 levels with one example each, explain how a classification can change, and how the functional/dysfunctional survey works.

TYPE: THEORY / MODEL · ~12 MARKS

REVEAL MODEL ANSWER

✓ MODEL ANSWER

The **Kano model** (Prof. Dr. Noriaki Kano, from quality management) prioritises customer requests by linking an implemented feature to customer satisfaction. Its **5 levels**:

- **Basic needs (must)**: self-evident; only noticed when missing — e.g. a call-centre greeting.
- **Performance needs (should)**: satisfaction scales with fulfilment — e.g. hotline waiting time.
- **Delighter (could)**: unexpected, creates enthusiasm — e.g. live status tracking of a ticket.
- **Indifferent**: little impact — e.g. the hold-music song.
- **Reverse (don't)**: its presence lowers satisfaction — e.g. jargon in a hotline call.

A classification **changes** by **customer** (delighter for one = reverse for another), by **price/business model** (delighter on a budget product = basic on a premium one), and over **time** (delighters become basic needs). The **survey** is standardized (to remove interviewer impact), includes demographics, and asks a **functional** question (“how would you feel if it had MORE of...?”) and a **dysfunctional** one (“...if it had LESS/none of...?”); the class is read from the answer combination (e.g. *expect it / dislike it* = *basic*; *like it / dislike it* = *performance*; *like it / neutral* = *delighter*).

Q3

Mira orders a custom robot pet. (a) Why must a requirement be SMART, atomic and linked — give one failure example. (b) Why “specific” but not “too specific”? (c) What does RM do in tier-down and in validation/verification?

TYPE: APPLICATION · ~8 MARKS

REVEAL MODEL ANSWER

✓ MODEL ANSWER

(a) A requirement must be **SMART** (Specific, Measurable, Achievable, Reasonable, Time-Bound), **atomic** (one requirement per sentence), in **active voice**, free of weak words, and **linked** to its request, mother/child & neighbour requirements, functions, test items and interfaces. **Failure example:** a bridge built from Germany (North-Sea zero) and Switzerland (Mediterranean zero) had a **28 cm step** because the height references were not linked — and the correction was even applied in the wrong direction.

(b) Natural language is **ambiguous** (“I saw the man on the mountain with the telescope” has many readings), so a requirement must be specific/testable (“efficient” → state 80%) — but **not too specific:** specify the *problem*, not the *solution*. Example: asked to reduce a bicycle's shipping damage, the team made a box that **looks like a TV**, cutting damage sharply — a solution that over-specification would have blocked.

(c) In **tier-down**, RM breaks system requirements to component level; the receiver must **accept, reject or request a change**, and only owns what it can build. In **validation/verification**, RM does **not** run tests but **tracks** whether each testable requirement was successfully confirmed.

One-screen cheat-card

Skeleton	Collect·Assess·Develop·Tier-down·Verify·Changes	basket→scale→cards→ladder→checklist→changes
Requirement	exact card for ONE property, all conditions	not a vague wish
Why RM	one common picture; targets wrong → product wrong	the tree-swing cartoon
Two docs	PRD=Lastenheft (customer, what) · Impl.Spec=Pflichtenheft (supplier, how)	Red Request · Green Guarantee
Collect	customer FIRST, then law/standards; internal = LeLe etc.	fewer secrets per extra interviewee
LeLe rule	solution ≠ negation; anchor a concrete action	“test ≥2 wks early, in the plan”
Kano 5	basic·performance·delighter·indifferent·reverse	bread·dial·gift·grey box·stink bomb
Kano shifts	by customer · by price · over time	delighter → basic over time
Classify	Product / Process / External · Functional / NFR	What / How / Rest
Develop	atomic, active, no weak words, testable, linked	one card, one idea
SMART	Specific·Measurable·Achievable·Reasonable·Time-bound	brain in a smart-watch
Specific	language ambiguous → IF-THEN; not too specific	telescope · bicycle-in-a-TV-box
Linked	request·mother/child·neighbour·function·test·interface	the 28 cm bridge step
System Wheel	hub(mech/EE/sw) + 7 DNA + 10 extended	wagon wheel
Tier down	accept / reject / feedback; own what you can build	bouncer at each door
Verify (5)	RM tracks ticks, doesn't test	checklist
ReqIF	XML, ProSTEP iViP, tool↔tool & cust↔supplier	universal plug